
MicroCloud

Release 0.99

Canonical Group Ltd

Jul 03, 2024

CONTENTS

1	In this documentation	3
2	Project and community	5
2.1	Tutorial	5
2.2	How-to guides	5
2.3	Reference	14
2.4	Explanation	15

MicroCloud is the easiest way to get a fully highly available LXD cluster up and running.

The MicroCloud snap automatically configures [LXD](#), [Ceph](#), and [OVN](#) across a set of servers. MicroCloud relies on mDNS to automatically detect other servers on the network, making it possible to set up a complete cluster by running a single command on one of the machines.

This way, MicroCloud creates a small footprint cluster of compute nodes with distributed storage and secure networking, optimised for repeatable, reliable remote deployments.

MicroCloud is aimed at edge computing, and anyone in need of a small-scale private cloud.

IN THIS DOCUMENTATION

Tutorial

Start here: a hands-on introduction to MicroCloud for new users

How-to guides

Step-by-step guides covering key operations and common tasks

Reference

Technical information - specifications, APIs, architecture

Explanation

Discussion and clarification of key topics

PROJECT AND COMMUNITY

MicroCloud is a member of the Ubuntu family. It's an open source project that warmly welcomes community projects, contributions, suggestions, fixes and constructive feedback.

- [MicroCloud snap](#)
- [Contribute](#)
- [Get support](#)
- [Announcement](#)
- [Thinking about using MicroCloud for your next project? Get in touch!](#)

2.1 Tutorial

MicroCloud is quick to set up. Once *installed*, you can start using MicroCloud in the same way as a regular LXD cluster.

If you are already familiar with LXD, you are good to go! Otherwise, see the [LXD documentation](#) to get started.

The following video shows how to install and use MicroCloud:

2.2 How-to guides

These how-to guides cover key operations and processes in MicroCloud.

2.2.1 How to install MicroCloud

To install MicroCloud, install all required *Snaps* on all machines that you want to include in your cluster.

To do so, enter the following command on all machines:

```
sudo snap install lxd microceph microovn microcloud
```

Note: Make sure to install the same version of the snaps on all machines.

If you don't want to use MicroCloud's full functionality, you can install only some of the snaps. However, this is not recommended.

2.2.2 How to manage the snaps

MicroCloud is distributed as a *snap*. The benefit of packaging MicroCloud as a snap is that it makes it possible to include the required dependencies, and that it allows MicroCloud to be installed on many different Linux distributions. The snap ensures that MicroCloud runs in a consistent environment.

Because MicroCloud uses a set of *other snaps*, you must make sure to have suitable versions of these snaps installed on all machines of your MicroCloud cluster. The installed snap versions must be compatible with one another, and for each of the snaps, the same version must be installed on all machines.

Choose the right channel and track

Snaps come with different channels that define which release of a snap is installed and tracked for updates. See [Channels and tracks](#) in the snap documentation for detailed information.

MicroCloud currently provides only the `latest` track.

Tip: In general, you should use the `latest/stable` channel for all snaps required to run MicroCloud.

However, since MicroCloud is not ready for production yet, you might want to use the `latest/edge` channel of the MicroCloud snap to benefit from the latest bug fixes.

When installing a snap, specify the channel as follows:

```
sudo snap install <snap_name> --channel=<channel>
```

For example:

```
sudo snap install microcloud --channel=latest/stable
```

To see all available channels of a snap, run the following command:

```
snap info <snap_name>
```

Control updates

By default, snaps are updated automatically. In the case of MicroCloud, this can be problematic because the related snaps must always use compatible versions, and because all machines of a cluster must use the same version of each snap.

Therefore, you should schedule your updates and make sure that all cluster members are in sync regarding the snap versions that they use.

Schedule updates

There are two methods for scheduling when your snaps should be updated:

- You can hold snap updates for a specific time, either for specific snaps or for all snaps on your system. After the duration of the hold, or when you remove the hold, your snaps are automatically refreshed.
- You can specify a system-wide refresh window, so that snaps are automatically refreshed only within this time frame. Such a refresh window applies to all snaps.

Hold updates

You can hold snap updates for a specific time or forever, for all snaps or for specific snaps.

Which strategy to choose depends on your use case. If you want to fully control updates to your MicroCloud setup, you should put a hold on all related snaps until you decide to update them.

Enter the following command to indefinitely hold all updates to the snaps needed for MicroCloud:

```
sudo snap refresh --hold lxd microceph microovn microcloud
```

When you choose to update your installation, use the following commands to remove the hold, update the snaps, and hold the updates again:

```
sudo snap refresh --unhold lxd microceph microovn microcloud
sudo snap refresh lxd microceph microovn microcloud --cohort="+"
sudo snap refresh --hold lxd microceph microovn microcloud
```

See [Hold refreshes](#) in the snap documentation for detailed information about holding snap updates.

Specify a refresh window

Depending on your setup, you might want your snaps to update regularly, but only at specific times that don't disturb normal operation.

You can achieve this by specifying a refresh timer. This option defines a refresh window for all snaps that are installed on the system.

For example, to configure your system to update snaps only between 8:00 am and 9:00 am on Mondays, set the following option:

```
sudo snap set system refresh.timer=mon,8:00-9:00
```

You can use a similar mechanism (setting `refresh.hold`) to hold snap updates as well. However, in this case the snaps will be refreshed after 90 days, irrespective of the value of `refresh.hold`.

See [Control updates with system options](#) in the snap documentation for detailed information.

Keep cluster members in sync

The cluster members that are part of the MicroCloud deployment must always run the same version of the snaps. This means that when the snaps on one of the cluster members are refreshed, they must also be refreshed on all other cluster members before MicroCloud is operational again.

Snaps updates are delivered as [progressive releases](#), which means that updated snap versions are made available to different machines at different times. This method can cause a problem for cluster updates if some cluster members are refreshed to a version that is not available to other cluster members yet.

To avoid this problem, use the `--cohort="+"` flag when refreshing your snaps:

```
sudo snap refresh lxd microceph microovn microcloud --cohort="+"
```

This flag ensures that all machines in a cluster see the same snap revision and are therefore not affected by a progressive rollout.

Use a Snap Store Proxy

If you manage a large MicroCloud deployment and you need absolute control over when updates are applied, consider installing a Snap Store Proxy.

The Snap Store Proxy is a separate application that sits between the snap client command on your machines and the snap store. You can configure the Snap Store Proxy to make only specific snap revisions available for installation.

See the [Snap Store Proxy documentation](#) for information about how to install and register the Snap Store Proxy.

After setting it up, configure the snap clients on all cluster members to use the proxy. See [Configuring snap devices](#) for instructions.

You can then configure the Snap Store Proxy to override the revisions for the snaps that are needed for MicroCloud:

```
sudo snap-proxy override lxd <channel>=<revision>
sudo snap-proxy override microceph <channel>=<revision>
sudo snap-proxy override microovn <channel>=<revision>
sudo snap-proxy override microcloud <channel>=<revision>
```

2.2.3 How to initialise MicroCloud

The *initialisation process* bootstraps the MicroCloud cluster. You run the initialisation on one of the machines, and it configures the required services on all machines.

During the initialisation process, you are prompted for information about your machines and how you want to set them up. The questions that you are asked might differ depending on your setup; for example, if you do not have the MicroOVN snap installed, you will not be prompted to configure your network, and if your machines don't have local disks, you will not be prompted to set up local storage.

The following instructions show the full initialisation process.

Tip: During initialisation, MicroCloud displays tables of entities to choose from.

To select specific entities, use the Up and Down keys to choose a table row and select it with the Space key. To select all rows, use the Right key. You can filter the table rows by typing one or more characters.

When you have selected the required entities, hit Enter to confirm.

Complete the following steps to initialise MicroCloud:

1. On one of the machines, enter the following command:

```
sudo microcloud init
```

2. Select the IP address that you want to use for MicroCloud's internal traffic. MicroCloud automatically detects the available addresses (IPv4 and IPv6) on the existing network interfaces and displays them in a table.

You must select exactly one address.

3. Decide if you want to limit the search for other machines.

If you accept the default (yes), MicroCloud will automatically detect machines in the local subnet. Otherwise, it will detect all available machines, which might include duplicates (if machines are available both on IPv4 and on IPv6).

See [Automatic server detection](#) for more information.

4. Select the machines that you want to add to the MicroCloud cluster.

MicroCloud displays all machines that it detects. Make sure that all machines that you select have the required snaps installed.

5. Select whether you want to set up local storage.

Note: To set up local storage, each machine must have a local disk. The disks must not contain any partitions.

If you choose **yes**, configure the local storage:

1. Select the disks that you want to use for local storage.

You must select exactly one disk from each machine.

2. Select whether you want to wipe any of the disks. Wiping a disk will destroy all data on it.

6. Select whether you want to set up distributed storage (using MicroCeph).

Note: To set up distributed storage, you need at least three additional disks on at least three different machines. The disks must not contain any partitions.

If you choose **yes**, configure the distributed storage:

1. Select the disks that you want to use for distributed storage.

You must select at least three disks.

2. Select whether you want to wipe any of the disks. Wiping a disk will destroy all data on it.

7. Select whether you want to set up distributed networking (using MicroOVN).

If you choose **yes**, configure the distributed networking:

1. Select the network interfaces that you want to use.

You must select one network interface per machine.

2. If you want to use IPv4, specify the IPv4 gateway on the uplink network (in CIDR notation) and the first and last IPv4 address in the range that you want to use with LXD.

3. If you want to use IPv6, specify the IPv6 gateway on the uplink network (in CIDR notation).

8. MicroCloud now starts to bootstrap the cluster. Monitor the output to see whether all steps complete successfully. See [Bootstrapping process](#) for more information.

Once the initialisation process is complete, you can start using MicroCloud.

Here's an example of the full initialisation process:

```
user@micro01:~$ sudo microcloud init          Select an address for MicroCloud's internal
traffic:Space to select; enter to confirm; type to filter results.Up/down to move; right
to select all; left to select none. +-----+-----+-----+ | ADDRESS | IFACE |
+-----+-----+-----+ [ ] | 203.0.113.169 | eth0 |> [X] | 2001:db8:d:100::169
| eth0 | +-----+-----+-----+ Using address "2001:db8:d:100::169" for
MicroCloud Limit search for other MicroCloud servers to 2001:db8:d:100::169/64?
(yes/no) [default=yes]: yesScanning for eligible servers ...Space to select; enter
to confirm; type to filter results.Up/down to move; right to select all; left to
select none. +-----+-----+-----+ | NAME | IFACE | ADDR |
+-----+-----+-----+> [x] | micro03 | eth0 | 2001:db8:d:100::171
| [x] | micro02 | eth0 | 2001:db8:d:100::170 | +-----+-----+-----+
```

Selected "micro03" at "2001:db8:d:100::171" Selected "micro02" at "2001:db8:d:100::170"
 Would you like to set up local storage? (yes/no) [default=yes]: yesSelect
 exactly one disk from each cluster member:Space to select; enter to confirm; type
 to filter results.Up/down to move; right to select all; left to select none.

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| LOCATION | MODEL | CAPACITY | TYPE | PATH | +-----+-----+-----+-----+
| [ ] | micro01 | Samsung SSD 870 EVO 500GB | 465.76GiB | sata | /dev/disk/by-id/
| wwn-0x5002538fc2b211bd | [x] | micro01 | Samsung SSD 980 250GB | 232.89GiB |
| nvme | /dev/disk/by-id/nvme-eui.002538dc21405ac8 | [ ] | micro02 | Samsung SSD
| 870 EVO 500GB | 465.76GiB | sata | /dev/disk/by-id/wwn-0x5002538fc2b2109c |
| [x] | micro02 | Samsung SSD 980 250GB | 232.89GiB | nvme | /dev/disk/by-id/
| nvme-eui.002538dc21405ad7 | [ ] | micro03 | Samsung SSD 870 EVO 500GB | 465.
| 76GiB | sata | /dev/disk/by-id/wwn-0x5002538fc2b2162d |> [x] | micro03 | Samsung
| SSD 980 250GB | 232.89GiB | nvme | /dev/disk/by-id/nvme-eui.002538dc21405aca |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Select which disks to wipe:Space to select; enter to confirm; type to
 filter results.Up/down to move; right to select all; left to select none.

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| LOCATION | MODEL | CAPACITY | TYPE | PATH | +-----+-----+-----+-----+
| [x] | micro01 | Samsung SSD 980 250GB | 232.89GiB | nvme | /dev/disk/by-id/
| nvme-eui.002538dc21405ac8 | [x] | micro02 | Samsung SSD 980 250GB | 232.89GiB
| | nvme | /dev/disk/by-id/nvme-eui.002538dc21405ad7 | [x] | micro03 | Samsung
| SSD 980 250GB | 232.89GiB | nvme | /dev/disk/by-id/nvme-eui.002538dc21405aca |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Using "/dev/disk/by-id/nvme-eui.002538dc21405ac8" on "micro01" for local storage
 pool Using "/dev/disk/by-id/nvme-eui.002538dc21405ad7" on "micro02" for local
 storage pool Using "/dev/disk/by-id/nvme-eui.002538dc21405aca" on "micro03" for local
 storage pool Would you like to set up distributed storage? (yes/no) [default=yes]:
 yesSelect from the available unpartitioned disks:Space to select; enter to confirm;
 type to filter results.Up/down to move; right to select all; left to select none.

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| LOCATION | MODEL | CAPACITY | TYPE | PATH | +-----+-----+-----+-----+
| [x] | micro01 | Samsung SSD 870 EVO 500GB | 465.76GiB | sata | /dev/disk/by-id/
| wwn-0x5002538fc2b211bd | [x] | micro02 | Samsung SSD 870 EVO 500GB | 465.76GiB
| | sata | /dev/disk/by-id/wwn-0x5002538fc2b2109c | [x] | micro03 | Samsung SSD
| 870 EVO 500GB | 465.76GiB | sata | /dev/disk/by-id/wwn-0x5002538fc2b2162d |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Select which disks to wipe:Space to select; enter to confirm; type to
 filter results.Up/down to move; right to select all; left to select none.

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| LOCATION | MODEL | CAPACITY | TYPE | PATH | +-----+-----+-----+-----+
| [x] | micro01 | Samsung SSD 870 EVO 500GB | 465.76GiB | sata | /dev/disk/by-id/
| wwn-0x5002538fc2b211bd | [x] | micro02 | Samsung SSD 870 EVO 500GB | 465.76GiB
| | sata | /dev/disk/by-id/wwn-0x5002538fc2b2109c | [x] | micro03 | Samsung SSD
| 870 EVO 500GB | 465.76GiB | sata | /dev/disk/by-id/wwn-0x5002538fc2b2162d |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Using 1 disk(s) on "micro02" for remote storage pool Using 1 disk(s) on "micro03" for
 remote storage pool Using 1 disk(s) on "micro01" for remote storage pool Configure
 distributed networking? (yes/no) [default=yes]: yesSelect exactly one network interface
 from each cluster member:Space to select; enter to confirm; type to filter results.Up/
 down to move; right to select all; left to select none. +-----+-----+-----+

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| LOCATION | IFACE | TYPE | +-----+-----+-----+> [x] | micro02 | eth0.
| 200 | vlan | [x] | micro03 | eth0.200 | vlan | [x] | micro01 | eth0.200 | vlan |
+-----+-----+-----+-----+-----+-----+-----+-----+
Using "eth0.200" on "micro03" for OVN uplink Using "eth0.
```

```

200" on "micro01" for OVN uplink Using "eth0.200" on "micro02" for OVN uplink Specify
the IPv4 gateway (CIDR) on the uplink network (empty to skip IPv4): 192.0.2.1/24Specify
the first IPv4 address in the range to use with LXD: 192.0.2.100Specify the last IPv4
address in the range to use with LXD: 192.0.2.254Specify the IPv6 gateway (CIDR) on the
uplink network (empty to skip IPv6): 2001:db8:d:200::1/64 Initializing a new cluster
Local MicroCloud is ready Local LXD is ready Local MicroOVN is ready Local MicroCeph is
readyAwaiting cluster formation ... Peer "micro02" has joined the cluster Peer "micro03"
has joined the clusterCluster initialization is completeMicroCloud is ready

```

2.2.4 How to add a machine

If you want to add a machine to the MicroCloud cluster after the initialisation, use the **microcloud add** command:

```
sudo microcloud add
```

Answer the prompts to add the machine. Alternatively, you can add the `--auto` flag to accept the default configuration instead of an interactive setup. You can also add the `--wipe` flag to automatically wipe any disks you add to the cluster.

2.2.5 How to get support

MicroCloud is not yet ready for production.

To benefit from the latest bug fixes, you should use the `latest/edge` snap channel for now.

Support and community

The following channels are available for you to interact with the MicroCloud community:

- You can file bug reports and feature requests as [GitHub issues](#).
- To ask questions, go to the MicroCloud section of our [discussion forum](#).

Commercial support

Commercial support for MicroCloud is available through [Ubuntu Pro](#).

Documentation

See the [MicroCloud documentation](#) for official product documentation.

You can find additional resources on the [website](#) and in the [discussion forum](#).

2.2.6 How to contribute to MicroCloud

The MicroCloud team appreciates contributions to the project, through pull requests, issues on the GitHub repository, or discussions or questions on the forum.

Check the following guidelines before contributing to the project.

Code of Conduct

When contributing, you must adhere to the Code of Conduct. MicroCloud follows the [Ubuntu Code of Conduct](#).

License and copyright

By default, any contribution to this project is made under the AGPL-3.0 license. See the [license](#) in the MicroCloud GitHub repository for detailed information.

All contributors must sign the [Canonical contributor licence agreement](#), which gives Canonical permission to use the contributions. The author of a change remains the copyright holder of their code (no copyright assignment).

Pull requests

Propose your changes to this project as pull requests on [GitHub](#).

Proposed changes will then go through review there and once approved, be merged in the main branch.

Commit structure

Use separate commits for every logical change, and for changes to different components. Prefix your commits with the component that is affected, using the code tree structure. For example, prefix a commit that updates the MicroCloud service with `microcloud/service:`.

This structure makes it easier for contributions to be reviewed and also greatly simplifies the process of porting fixes to other branches.

Developer Certificate of Origin

To improve tracking of contributions to this project we use the DCO 1.1 and use a “sign-off” procedure for all changes going into the branch.

The sign-off is a simple line at the end of the explanation for the commit which certifies that you wrote it or otherwise have the right to pass it on as an open-source contribution.

Developer Certificate of Origin
Version 1.1

Copyright (C) 2004, 2006 The Linux Foundation and its contributors.
660 York Street, Suite 102,
San Francisco, CA 94110 USA

Everyone is permitted to copy and distribute verbatim copies of this
license document, but changing it is not allowed.

Developer's Certificate of Origin 1.1

By making a contribution to this project, I certify that:

- (a) The contribution was created in whole or in part by me and I
have the right to submit it under the open source license
indicated in the file; or

(continues on next page)

(continued from previous page)

- (b) The contribution **is** based upon previous work that, to the best of my knowledge, **is** covered under an appropriate **open** source license **and** I have the right under that license to submit that work **with** modifications, whether created **in** whole **or in** part by me, under the same **open** source license (unless I am permitted to submit under a different license), **as** indicated **in** the file; **or**
- (c) The contribution was provided directly to me by some other person who certified (a), (b) **or** (c) **and** I have **not** modified it.
- (d) I understand **and** agree that this project **and** the contribution are public **and** that a record of the contribution (including **all** personal information I submit **with** it, including my sign-off) **is** maintained indefinitely **and** may be redistributed consistent **with** this project **or** the **open** source license(s) involved.

An example of a valid sign-off line is:

```
Signed-off-by: Random J Developer <random@developer.org>
```

Use a known identity and a valid e-mail address, and make sure that you have signed the [Canonical contributor licence agreement](#). Sorry, no anonymous contributions are allowed.

We also require each commit be individually signed-off by their author, even when part of a larger set. You may find `git commit -s` useful.

Contribute to the documentation

We want MicroCloud to be as easy and straight-forward to use as possible. Therefore, we aim to provide documentation that contains the information that users need to work with MicroCloud, that covers all common use cases, and that answers typical questions.

You can contribute to the documentation in various different ways. We appreciate your contributions!

Typical ways to contribute are:

- Add or update documentation for new features or feature improvements that you contribute to the code. We'll review the documentation update and merge it together with your code.
- Add or update documentation that clarifies any doubts you had when working with the product. Such contributions can be done through a pull request or through a post in the [discussion forum](#). Tutorials and other documentation posted in the forum will be considered for inclusion in the docs (through a link or by including the actual content).
- To request a fix to the documentation, open a [documentation issue](#) on GitHub. We'll evaluate the issue and update the documentation accordingly.
- Post a question or a suggestion on the [discussion forum](#). We'll monitor the posts and, if needed, update the documentation accordingly.

Documentation framework

MicroCloud’s documentation is built with [Sphinx](#) and hosted on [Read the Docs](#).

It is written in [reStructuredText](#). For syntax help and guidelines, see our [reStructuredText style guide](#). The repository also contains a [documentation cheat sheet](#), which can be helpful as a quick reference.

For structuring, the documentation uses the [Diátaxis](#) approach.

Build the documentation

To build the documentation, go to the `doc` folder of the repository and run **make html**. This command installs the required tools and renders the output to the `doc/_build/` directory. Subsequent builds only process files that have changed. To run a clean build of all files, but without re-installing all tools, run **make clean-doc html**.

Before opening a pull request, make sure that the documentation builds without any warnings (warnings are treated as errors). To preview the documentation locally, run **make serve** and go to <http://localhost:8000> to view the rendered documentation.

When you open a pull request, a preview of the documentation output is built automatically. To see the output, view the details for the `docs/readthedocs.com:canonical-microcloud` check on the pull request.

Automatic documentation checks

GitHub runs automatic checks on the documentation to verify the spelling, the validity of links, and the use of inclusive language.

You can (and should!) run these tests locally as well with the following commands:

- Check the spelling: **make spelling**
- Check the validity of links: **make linkcheck**
- Check for inclusive language: **make woke**

2.3 Reference

The reference material in this section provides technical descriptions of MicroCloud.

2.3.1 Requirements

Hardware requirements

MicroCloud requires a minimum of three machines. It supports up to 50 machines.

Each machine must have at least 8 GiB of RAM (more depending on the connected disks). You can mix different processor architectures within the same MicroCloud cluster.

If you want to add further machines after the initial initialisation, you can use the **microcloud add** command.

To use local storage, each machine requires a local disk. To use distributed storage, at least three additional disks (not only partitions) for use by Ceph are required, and these disks must be on at least three different machines.

Networking requirements

For networking, MicroCloud requires a dedicated network interface and an uplink network that is an actual L2 subnet. See [Networking](#) for more information.

The IP addresses of the machines must not change after installation, so DHCP is not supported.

Software requirements

MicroCloud requires snapd version 2.59 or newer.

2.3.2 Snaps

To run MicroCloud, you must install the following snaps:

- [MicroCloud snap](#)
- [LXD snap](#)
- [MicroCeph snap](#)
- [MicroOVN snap](#)

2.4 Explanation

The explanatory guides in this section introduce you to the concepts used in MicroCloud and help you understand how things fit together.

2.4.1 About MicroCloud

The MicroCloud snap drives three other snaps (LXD, MicroCeph, and MicroOVN), enabling automated deployment of a highly available LXD cluster for compute with Ceph as the storage driver and OVN as the managed network.

During initialisation, MicroCloud detects the other servers and then prompts you to add disks to Ceph and configure the networking setup.

At the end of this, you'll have an OVN cluster, a Ceph cluster, and a LXD cluster. LXD itself will have been configured with both networking and storage suitable for use in a cluster.

LXD cluster

MicroCloud sets up a LXD cluster. You can use the **microcloud cluster** command to show information about the cluster members, or to remove specific members.

Apart from that, you can use LXD commands to manage the cluster. See [Clustering](#) in the LXD documentation for more information.

Networking

By default, MicroCloud uses MicroOVN for networking, which is a minimal implementation of OVN (Open Virtual Network). If you decide to not use MicroOVN, MicroCloud falls back on the Ubuntu fan for basic networking.

MicroOVN requires an uplink network that is an actual L2 subnet (which is usually not the case in a virtual cloud environment). In addition, MicroOVN requires its own dedicated network interface, for example, a dedicated physical network interface, a VLAN, or a virtual function on an SR-IOV (Single root I/O virtualisation)-capable network interface.

See [OVN network](#) in the LXD documentation for more information.

Storage

You have two options for storage in MicroCloud: local storage or distributed storage.

Local storage is faster, but less flexible and not fail-safe. To use local storage, each machine in the cluster requires a local disk. Disk sizes can vary.

For distributed storage, MicroCloud uses MicroCeph, which is a lightweight way of deploying a Ceph cluster. To use distributed storage, you must have at least three disks (attached to at least three different machines).

2.4.2 About the initialisation process

See [How to initialise MicroCloud](#) for instructions on how to set up MicroCloud.

Automatic server detection

MicroCloud uses mDNS (multicast DNS) to automatically detect other servers on the network. This method works in physical networks, but it is usually not supported in a cloud environment.

By default, the scan is limited to the local subnet.

MicroCloud will display all servers that it detects, and you can then select the ones you want to add to the MicroCloud cluster.

Bootstrapping process

After you provide the required information to [initialise MicroCloud](#), MicroCloud starts bootstrapping the cluster.

The bootstrapping process consists of the following steps:

1. MicroCloud initialises the first server (the one where you run the **microcloud init** command) and creates the MicroCloud cluster.
2. MicroCloud creates the LXD cluster, the OVN cluster, and the Ceph cluster.
3. MicroCloud issues join tokens for the other servers that are to be added to the cluster.

4. MicroCloud sends the join tokens over the network to the other servers.
5. The other servers initialise their services and join the MicroCloud cluster, the OVN cluster, and the Ceph cluster.

This step of forming the cluster can take several minutes, mainly because of the initialisation of MicroCeph and adding disks to the Ceph cluster.

6. When the cluster is formed, MicroCloud configures LXD. It sets up networking and storage pools and configures the default profile to use the created OVN network and the distributed storage (if available).

After the initialisation is complete, you can look at the LXD configuration to confirm the setup.

```
user@micro01:~$ lxc cluster list+-----+-----+-----+-----+-----+-----+
NAME | URL | ROLES | ARCHITECTURE | FAILURE DOMAIN | DESCRIPTION | STATE | MESSAGE
+-----+-----+-----+-----+-----+-----+-----+-----+
micro01 | https://[2001:db8:d:100::169]:8443 | database-leader | aarch64
| default | | ONLINE | Fully operational | | | database | | | |
+-----+-----+-----+-----+-----+-----+-----+-----+
micro02 | https://[2001:db8:d:100::170]:8443 | database | aarch64 | default | | ONLINE |
Fully operational |
+-----+-----+-----+-----+-----+-----+-----+-----+
micro03 | https://[2001:db8:d:100::171]:8443 | database | aarch64 | default | | ONLINE |
Fully operational |
+-----+-----+-----+-----+-----+-----+-----+-----+
user@micro01:~$ lxc storage list+-----+-----+-----+-----+-----+-----+
NAME | DRIVER | DESCRIPTION | USED BY | STATE |
+-----+-----+-----+-----+-----+-----+-----+-----+
local | zfs | Local storage on ZFS | 10 | CREATED |
remote | ceph | Distributed storage on Ceph | 7 | CREATED |
+-----+-----+-----+-----+-----+-----+-----+-----+
user@micro01:~$ lxc network list+-----+-----+-----+-----+-----+-----+
NAME | TYPE | MANAGED | IPV4 | IPV6 | DESCRIPTION | USED BY | STATE
+-----+-----+-----+-----+-----+-----+-----+-----+
UPLINK | physical | YES | | | 2 | CREATED |
+-----+-----+-----+-----+-----+-----+-----+-----+
default | ovn | YES | 10.123.123.1/24 | fd42:1234:1234:1234::1/64 | | 5 | CREATED
+-----+-----+-----+-----+-----+-----+-----+-----+
eth0 | physical | NO | | | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+
eth0.200 | vlan | NO | | | 1 |
+-----+-----+-----+-----+-----+-----+-----+-----+
user@micro01:~$ lxc profile show default          config: {}description: ""devices:eth0:
name: eth0 network: default type: nicroot: path: / pool: remote type: diskname:
defaultused_by: []
```